Görüntü İşleme
Proje Ara Rapor Formu

“Piksel Tabanlı Manuel Görüntü Editörü Uygulaması”

Giriş

Dijital görüntü işleme, günümüzde otonom sistemlerden tıbbi görüntülemeye kadar pek çok ileri teknoloji disiplininin temelini oluşturmaktadır. Biz bu proje kapsamında, dijital bir görselin piksel düzeyinde nasıl değiştirilebileceğini anlamak ve temel görüntü işleme algoritmalarını hazır fonksiyon kütüphanelerine başvurmadan kendimiz oluşturmaya çalıştık. Yazılım, bir görüntüyü (M * N) boyutlarında bir matris veri yapısı olarak ele almak ve her bir hücredeki renk yoğunluk değerlerini (Red, Green, Blue) matematiksel operatörlerle yeniden
oluşturuyor.[1, 4] Projemizi hazırladığımız süreç boyunca , ham verinin iyileştirilmesi ve segmentasyonu için kritik öneme sahip olan; Gri Seviye Dönüşümü, Binary (Eşikleme) Segmentasyonu, Renk Uzayı Analizi (Kanal İzolasyonu) gibi temel işlemlerin yanı sıra; Görüntü Döndürme, Kırpma ve Yaklaştırma/Uzaklaştırma (Zooming) gibi geometrik dönüşümler bizzat geliştirmeye çalıştık.

Çalışmamızda , görüntü işlemenin arka planındaki lineer cebir ilkelerini uygulamaya taşımayı ve görsel veriler üzerinde tam denetim sağlayan bir altyapı oluşturmayı amaçladık.

Fotoğraf Editör uygulaması kapsamında gerçekleştireceğimiz işlemler şunlar:

- Gri Dönüşüm
- Binary (ikili) Dönüşüm
- Görüntü Döndürme ve Kırpma (koordinat dönüşümleri kullanılarak manuel gerçekleştirme)
- Görüntü Yaklaştırma ve Uzaklaştırma (nearest neighbor interpolasyonu ile manuel ölçekleme)
- Renk Uzayı Analizi (RGB kanal ayrıştırma ve kanal bazlı işlemler)
- Histogram ve Orjinal Görüntü Histogramını Germe ve Genişletme
- İki resim arasında aritmetik işlemler (çıkarma, çarpma vb.)
- Kontrast Azaltma
- Konvolüsyon İşlemi (3x3 komşuluk matrisi kullanılarak manuel median ve mean filtreleme)
- Eşikleme İşlemleri (Çift Eşikleme)
- Kenar belirleme işlemleri (gradyan temelli maskeler kullanılarak manuel hesaplama)
- Görüntüye Gürültü Ekleme (Salt & Pepper) ve 3x3 komşuluk penceresi kullanılarak manuel mean ve median filtreleme ile gürültü azaltma
- Görüntüye Filtre Uygulanması (manuel olarak oluşturulan kernel matrisi ile motion blur)
- Morfolojik İşlemler (Genişleme, Aşınma, Açma,Kapama)

Not: Yukarıda belirttiğimiz tüm işlemlerde, herhangi bir hazır görüntü işleme fonksiyonu kullanmamaya , matematiksel hesaplamalar ile kendimiz oluşturmaya çalıştık.

Bu çalışmada ana amacımız , dijital bir görselin sadece bir fotoğraf değil, işlenebilir ve yönetilebilir bir sayısal veri kümesi olduğunu akademik bir çalışmayla ortaya koymaktı. Geliştirdiğimiz bu sistem ile kullanıcıya görsel veriler üzerinde doğrudan kontrol imkanı tanırken, görüntü işlemenin, uygulamalarla pekiştirilmesini sağlıyoruz.

Literatür Taraması

Kullandığımız yöntemler, literatürde yaygın olarak kullanılan temel görüntü işleme tekniklerine dayanıyor.

Histogram ve Kontrast Germe / Genişletme

Histogram Tanımı:
Dijital bir görüntüde histogram, her bir yoğunluk seviyesindeki (0–255) piksel sayısının dağılımını gösteren ayrık bir fonksiyondur. Histogram, görüntünün parlaklık dağılımını analiz etmek için kullanılan temel araçlardan biridir.[4, 5]

Projemizde histogram germe işlemini, görüntüdeki minimum ve maksimum piksel değerlerini belirleyerek, bu aralıkta lineer olarak yeniden ölçeklendirme yaparak kendimiz gerçekleştirdik.

Görsel Analiz:
Histogram analizi, görüntünün kontrast ve parlaklık özellikleri hakkında önemli bilgiler sunar. Bu analiz sayesinde görüntünün düşük kontrastlı mı yoksa yüksek kontrastlı mı olduğu anlaşılabilir.

Kontrast Germe (Contrast Stretching)
Kontrast germe işlemi, görüntüdeki piksel yoğunluklarını tüm dinamik aralığa (0'dan L−1L-1L−1'e) yayarak görüntünün görsel kalitesini ve detay görünürlüğünü artırmayı amaçlar. Bu işlem matematiksel olarak aşağıdaki şekilde ifade edilebilir:

$$I'(x,y) = \frac{I(x,y) - I_{min}}{I_{max} - I_{min}} \times (L-1)$$

Burada IminI_{min}Imin ve ImaxI_{max}Imax, görüntüdeki minimum ve maksimum piksel değerlerini temsil ediyor.

Global ve Yerel Yaklaşımlar:
Literatürde kontrast iyileştirme yöntemleri ikiye ayrılıyor:

- **Global yöntemler:** Tüm görüntüye tek bir dönüşüm uygulanır.
- **Yerel (Local) yöntemler:** Görüntü küçük bölgelere ayrılarak her bölgeye ayrı işlem uygulanır (örneğin CLAHE).

Kalite Metrikleri:
İyileştirilmiş görüntülerin başarısı, matematiksel ölçütlerle değerlendirilir. En yaygın kullanılan metrikler:

- **PSNR (Peak Signal-to-Noise Ratio)**
- **SSIM (Structural Similarity Index)**

Bu metrikler, orijinal ve işlenmiş görüntü arasındaki benzerliği ölçmek için kullanılır.

İki Resim Arasında Aritmetik İşlemler (Çıkarma ve Çarpma)

Piksel Tabanlı İşlemler:
Aritmetik işlemler, aynı boyuttaki iki görüntü arasında piksel bazlı (pixel-wise) matris işlemleri olarak gerçekleştirilir. Her piksel değeri, karşılık gelen piksel ile ayrı ayrı işlenir.

Görüntü Çıkarma (Image Subtraction):
İki görüntü arasındaki farkı belirlemek için kullanılır ve özellikle nesne takibi ile arka plan çıkarma (background subtraction) işlemlerinde önemli bir rol oynar.

$$I'(x,y) = I_1(x,y) - I_2(x,y)$$

Görüntü Çarpma (Image Multiplication):
Çarpma işlemi genellikle maskeleme (masking) amacıyla kullanılır. Belirli bölgeleri korumak veya vurgulamak için tercih edilir.

$$I'(x,y) = I_1(x,y) \times I_2(x,y)$$

Bu işlemleri, karşılık gelen piksel değerlerine doğrudan matematiksel işlemler uygulayarak oluşturduk .[1, 7]

Kontrast Azaltma

Dinamik Aralığın Sıkıştırılması:
Kontrast azaltma işlemi, piksel yoğunlukları arasındaki farkı azaltarak görüntünün daha yumuşak ve soluk görünmesini sağlar. Bu işlem, dinamik aralığın daraltılması ile gerçekleştirilir.

$$I'(x,y) = \alpha \cdot I(x,y) \quad (0 < \alpha < 1)$$

Parlaklık Kontrolü:
Bu işlem, görüntünün belirli bir parlaklık seviyesinde tutulmasını sağlar ve aşırı parlak bölgelerin dengelenmesine yardımcı olur.

Projemizde kontrast azaltma işlemini, piksel değerlerini belirli bir aralığa sıkıştırarak gerçekleştirdik.

Eşikleme İşlemleri (Çift Eşikleme - Double Thresholding)

Gelişmiş Kenar Belirleme:
Çift eşikleme yöntemi, pikselleri iki farklı eşik değerine göre sınıflandıran bir tekniktir. Bu yöntem özellikle kenar algılama algoritmalarında kullanılıyro.

Gürültü Filtreleme:

- Düşük eşik değerinin altındaki pikseller gürültü olarak bastırılır
- Yüksek eşik değerinin üzerindeki pikseller "güçlü kenar" olarak kabul edilir

Süreklilik ve Histerezis:
İki eşik değeri arasında kalan pikseller "zayıf kenar" olarak değerlendirilir. Bu pikseller yalnızca güçlü kenarlara bağlıysa korunur. Bu süreç, histerezis olarak adlandırılır ve kesintisiz kenarların elde edilmesini sağlar.

Literatür taraması yaparken kullandığımız bu yöntemlerin tamamı, projemizde herhangi bir hazır görüntü işleme fonksiyonu kullanmadan matematiksel modelleri baz alarak sıfırdan oluşturulmaya çalışılarak oluşturduk.

Kullanılan/Kullanılacak Teknolojiler

Projemizde, görüntü işlemede temel kavramları daha iyi öğrenmek ve algoritma mantığını kavrayabilmek amacıyla, hazır kütüphanelerin sunduğu görüntü işleme fonksiyonları (temel okuma-yazma işlemleri hariç) kullanmadık; algoritmaları tamamiyle kendimiz oluşturduk. Projemizde kullandığımız teknolojileri aşağıda teknik detaylarıyla açıkladık.

Python (Programlama Dili)

Python, açık kaynaklı, yüksek seviyeli ve yorumlanabilir bir programlama dilidir. Projede matris manipülasyonları ve piksel bazlı işlemlerin (nested loops) esnek bir şekilde uygulanabilmesi amacıyla ana geliştirme dili olarak tercih edilmiştir. Tüm görüntü işleme algoritmaları (gri dönüşüm, eşikleme, filtreleme, kenar bulma, morfoloji, geometrik dönüşümler) herhangi bir hazır fonksiyon kullanılmadan, matematiksel modellere dayanarak manuel olarak oluşturulmuştur.

Kullanılan kütüphaneler:

NumPy

- **Kullanım Amacı:** Görsellerin her bir pikselini (M*N) boyutlarında sayısal matrisler olarak yönetmek için kullandık.
- **Teknik Detay:** NumPy, yalnızca matris oluşturma (np.zeros) ve veri tipi yönetimi (uint8) için kullandık. Tüm filtreleme, gradyan, morfolojik işlemler ve kanal ayrıştırma işlemlerini, NumPy dizisine doğrudan piksel erişimi (img[i][j]) ile ve iç içe döngüler kullanarak **manuel olarak gerçekleştirdik**. Hazır filtreleme fonksiyonları veya konvolüsyon metotlarını kullanmadık.

Matplotlib (Pyplot)

- **Kullanım Amacı:** İşlenen sonuçların (Orijinal, Gri, Binary, Döndürülmüş vb.) tek bir arayüzde karşılaştırmalı olarak sunulması için kullandık.
- **Teknik Detay:** Matplotlib, işlenen görüntülerin karşılaştırmalı olarak görselleştirilmesi amacıyla kullandık. Orijinal ve işlenmiş görüntüler, np.hstack ve np.vstack ile birleştirerek tek panelde gösterdik. Matplotlib, yalnızca görselleştirme amacıyla görüntü işleme algoritmalarına katkı sağlamamaya çalıştık.

OpenCV (cv2)

- **Kısıtlama:** Projemizin özgünlüğünü korumak için bu kütüphane sadece "Girdi/Çıktı" (I/O) ile sınırladık.
- **Açıklama:** “cv2.rotate, cv2.resize veya cv2.threshold” gibi hazır metotları istenildiği üzere projeye dahil etmedik .OpenCV kütüphanesi yalnızca görüntü okuma (cv2.imread) ve yazma/gösterme (cv2.imwrite, cv2.imshow) için kullandık. Hiçbir hazır görüntü işleme fonksiyonu (resize, rotate, threshold, blur

vb.) projemizde kullanmadık. Tüm işlemleri piksel bazlı olarak kendimiz oluşturduk.

Tkinter

- **Kullanım Amacı:**
  Python'un standart GUI kütüphanesi olan tkinter, temel kullanıcı arayüzü bileşenlerini oluşturmak için kullanılacak (pencere, temel etkileşimler).
- **Açıklama:**
  Görsel işlemlere katkı sağlamayacak; kullanıcıdan dosya seçimi gibi görevlerde görev alacak.Arayüz kütüphaneleri yalnızca kullanıcı etkileşimi (dosya seçimi, buton, slider, progress bar) için kullandık. Görüntü işlemeye katkı sunmuyor.

customtkinter

- **Kullanım Amacı:**
  tkinter üzerine bileşenler ekleyen bu kütüphane, arayüzünü daha kullanıcı dostu ve şık hale getirmek amacıyla kullanıcaz.
- **Projede Kullanımı:**
  Frame, Button, Slider, ProgressBar gibi görsel bileşenler için kullanılacak. Görüntü işleme algoritmalarıyla bağlantısı yok.

Threading

- **Kullanım Amacı:** Konvolüsyon (Median, Mean) ve Kenar Bulma (Canny) gibi işlem yükü ağır olan algoritmalar çalışırken arayüzün donmasını engellemek amacıyla kullanmayı hedefledik .
- **Açıklama:** Ağır matematiksel hesaplamalar arka planda (background) çalıştırılırken, kullanıcı arayüzü etkileşime açık kalmaya devam edecek. Threading, yoğun matematiksel hesaplamaların arka planda çalıştırılması için kullandık. Bu sayede GUI donmadan kullanıcı etkileşimi sağlayabilecek.

Algoritmik Yaklaşım ve Metodoloji

Projemizde üm görüntü işleme algoritmaları, görüntünün iki boyutlu bir piksel matrisi olarak ele alıp geliştirmeye çalıştık. Görüntü, NumPy dizisi şeklinde temsil edilip her piksele img[i][j] indisleri ile doğrudan erişiliyor

Tüm işlemler, iç içe döngüler (nested loops) kullanılarak piksel bazlı olarak gerçekleşmekte. Her bir algoritma, ilgili matematiksel model doğrultusunda kendimizce oluşturuldu...

Gri Seviye Dönüşümü:

Her piksel için RGB değerleri kullanılarak ağırlıklı ortalamayı hesapladık. Bu işlemde parlaklık (luminance) modelini kullandk.

$$Gray(i,j) = 0.299 \cdot R(i,j) + 0.587 \cdot G(i,j) + 0.114 \cdot B(i,j)$$

Her piksel için R, G ve B değerleri bu formülle hesaplanıyor ve img[i][j] ile doğrudan yazılıyor.

Binary (Eşikleme):

Görüntü üzerindeki her pikseli, belirlenen eşik değeri ile karşılaştırdık ve sonuç görüntüsü ikili (0 veya 255) olacak şekilde oluşturduk.

$$g(i,j) = \begin{cases} 255 & f(i,j) > T \\ 0 & \text{diğer} \end{cases}$$

Eşik değeri T kullanılarak her piksel sınıflandırılır; sınır kontrolleri sayesinde indisler diziden taşmaz.

Histogram Germe:

Görüntüdeki minimum ve maksimum piksel değerleri tespit ettik, tüm piksel değerleri bu aralığa lineer olarak yeniden düzenledik.

Gürültü Ekleme:

Salt & Pepper gürültüsü, rastgele seçilen piksel koordinatlarına minimum (0) ve maksimum (255) değerler atanarak kendimizce oluşturduk.

Filtreleme (Mean ve Median):

Filtreleme işlemlerinde her piksel için komşuluk (3x3 pencere) dikkate almaya çalıştık. Mean filtrede komşu piksel değerlerinin ortalamasını alırkne, Median filtrede bu değerleri sıralayark ortanca değer seçtik.[3, 5]

$$g(i,j) = \frac{1}{9} \sum_{m=-1}^{1} \sum_{n=-1}^{1} f(i+m, j+n)$$

Mean: 3x3 komşuluk penceresindeki değerlerin ortalaması alınır:

Median: 3x3 komşuluk değerleri sıralanır, ortanca değer seçilir . Komşuluk penceresi sınırları kontrol edilerek indis taşmaları önlenir (boundary check).

Kenar Bulma:

Yoğunluk değişimlerini belirlemek amacıyla, x ve y yönlerinde gradyan hesaplayan maskeler (kernel) kullanılmıştır. Bu işlem, hazır Sobel fonksiyonu kullanmadan, manuel konvolüsyon işlemi ile gerçekleştirdik.X ve Y yönde gradyan maskesini uyguladık:

$$G_x = \sum_{m=-1}^{1} \sum_{n=-1}^{1} f(i+m, j+n) \cdot S_x[m+1][n+1]$$

$$G_y = \sum_{m=-1}^{1} \sum_{n=-1}^{1} f(i+m, j+n) \cdot S_y[m+1][n+1]$$

$$G(i,j) = \sqrt{G_x^2 + G_y^2}$$

Maskeleri kendimiz oluşturduk, hazır Sobel fonksiyonu kullanmadık.[2, 6]

Morfolojik İşlemler:

Genişleme (dilation) işleminde komşu piksellerin maksimum değeri, aşınma (erosion) işleminde ise minimum değeri alarak yeni piksel değerleri oluşturduk.[1, 3]
Dilation (Genişleme): komşu piksellerin maksimum değeri
Erosion (Aşınma): komşu piksellerin minimum değeri

Her piksel için 3x3 pencere uygulayıp; sınır kontrolleri ile taşmalar önlemeye çalıştık olabildiğince.

Geometrik Dönüşümler:

Döndürme ve ölçekleme işlemleri, koordinat dönüşümleri kullanarak gerçekleştirdik. Her pikselin yeni konumu matematiksel olarak hesaplayıp ve uygun şekilde yeni görüntü matrisi oluşturduk.[2, 7]

Piksel koordinatları matematiksel olarak yeni matrise eşlenir:

$$x' = \cos\theta \cdot x - \sin\theta \cdot y$$
$$y' = \sin\theta \cdot x + \cos\theta \cdot y$$

Ölçekleme (Zoom):

$$x' = x \cdot S_x, \quad y' = y \cdot S_y$$

Her yeni koordinat sınırlar içinde kontrol edilip ; taşma durumları engellenir bu sayede (boundary check).

Tüm algoritmalarda sınır (boundary) kontrolünü dikkate aalmaya çalışıp , görüntü kenarlarında taşma (out-of-bounds) durumları engelledik. Bundan dolayı komşuluk işlemlerinde yalnızca geçerli indisler değerlendirdik.

Sistem Akış Diyagramı (Flowchart) ve Açıklaması

Geliştirdiğimiz piksel tabanlı görüntü işleme uygulamasının genel çalışma prensibini, aşağıda verdiği,miz akış diyagramı ile gösterdik. Bu diyagram, sistemin giriş aşamasından başlayarak kullanıcı tarafından seçilen görüntü işleme algoritmasının uygulanması ve sonuçların elde edilmesine kadar geçen tüm süreci adım adım açıklıyor.

Sistem, ilk aşamada kullanıcıdan bir görüntü almasıyla başlıyor. Seçilen görüntü, OpenCV kütüphanesi kullanılarak okunuyor ve NumPy dizisi formatında belleğe aktarılıyor. Bu aşamada görüntü, üç boyutlu bir matris olarak temsil edilip her bir piksele doğrudan indeksleme yöntemi ile erişim sağlıyor.

Ardından kullanıcı, arayüz üzerinden gerçekleştirmek istediği görüntü işleme operasyonunu seçer. Bu seçim doğrultusunda sistem, ilgili algoritmayı tetikleyerek yalnızca seçilen işlemi görüntü üzerine uygular. Bu yaklaşım, pipeline (ardışık işlem) yerine kullanıcı kontrollü tek işlem prensibine dayanmasını hedefledik.

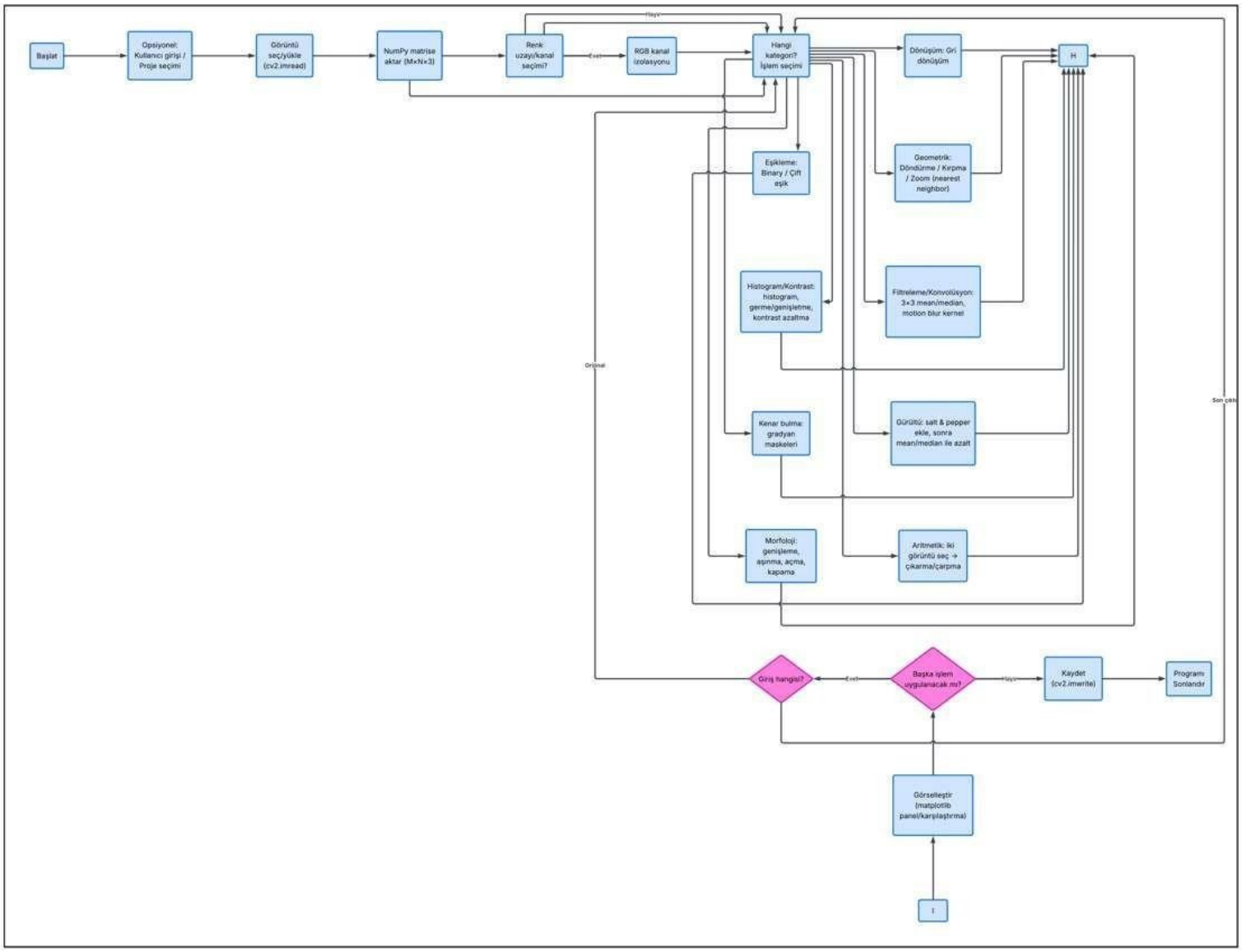
Seçilebilen işlemler; gri seviye dönüşümü, binary (eşikleme) işlemleri, histogram analizi ve kontrast iyileştirme, geometrik dönüşümler (döndürme, kırpma, ölçekleme), filtreleme (mean ve median), kenar belirleme, gürültü ekleme ve azaltma, morfolojik işlemler ve iki görüntü arasında aritmetik işlemler gibi temel görüntü işleme tekniklerini kapsıyor.

Her bir algoritmada, herhangi bir hazır görüntü işleme fonksiyonu kullanmadan, doğrudan piksel değerleri üzerinde kendimizin emeği ile oluşturuldu. Özellikle filtreleme, kenar bulma ve morfolojik işlemler sırasında komşuluk ilişkileri dikkate alınmış ve 3x3 pencere yapısını kullandık. Bu süreçte sınır kontrolü (boundary check) mekanizması ile görüntü kenarlarında oluşabilecek taşmalar engelledik.

Seçilen işlemin uygulanmasının ardından elde edilen yeni görüntü, kullanıcıya görsel olarak sunuluyor. Görselleştirme aşamasında Matplotlib kütüphanesi kullandik ki orijinal ve işlenmiş görüntüler karşılaştırmalı olarak gösterebilelim. Kullanıcı isteğine bağlı olarak elde edilen sonuç görüntüsü dosya olarak kaydedilebilmekte veya yeni bir işlem için süreç yeniden başlatılabilmekte bu sayede.

Sonuç olarak bu akış diyagramı ile sistemin modüler yapısını, kullanıcı etkileşimini ve görüntü işleme algoritmalarının çalışma sırasını açık bir şekilde ortaya koyduğumuz düşünüyrouz. Bu yapı sayesinde hem algoritmaların işleyişi şeffaf hale getirdik hem de kullanıcıya esnek ve kontrol edilebilir bir görüntü işleme ortamı sunduk.

Sistem Akış Diyagramı :



Veri Tabanı Tasarımı ve ER Diyagramı Açıklaması

Projemizin sürdürülebilirliğini sağlamak, gerçekleştirilen görüntü işleme adımlarını sistematik bir şekilde kayıt altına almak ve işlemlerin geriye dönük takibini sağlamak amacıyla bir Varlık-İlişki (ER) diyagramı tasarladıık. Bu diyagram, “Piksel Tabanlı Manuel Görüntü Editörü” yaklaşımına uygun olarak, görüntü verisi ile uygulanan işlemler arasındaki ilişkileri modelliyor.

Sistemde gerçekleştirilen her bir görüntü işleme operasyonu (gri seviye dönüşümü, eşikleme, histogram germe vb.), ISLEM varlığı altında tanımladık. Bu işlemlere ait parametreler (örneğin eşikleme işlemi için kullanılan eşik değeri TTT) ise PARAMETRE varlığı aracılığıyla saklıyor. Bu yapı sayesinde, algoritmaların hangi koşullar altında çalıştırıldığı kayıt altına alınarak işlemlerin tekrar üretilebilirliği sağlmaya çalıştık.

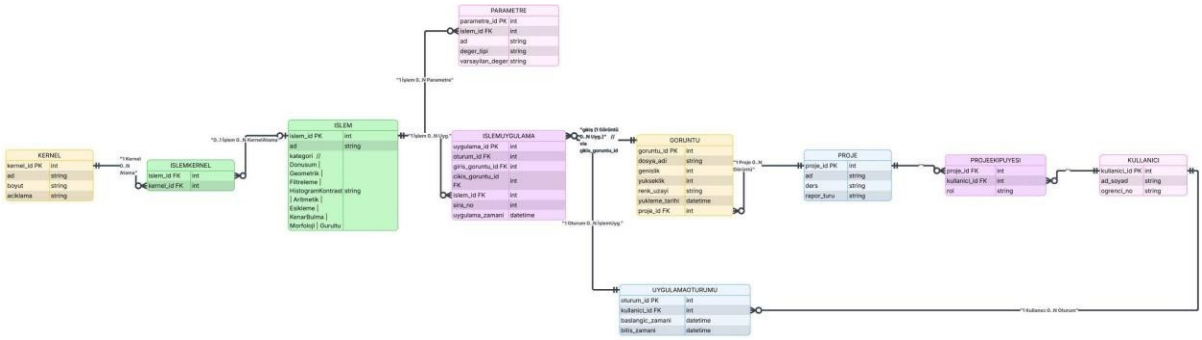
Filtreleme ve konvolüsyon tabanlı işlemler için, projemizde kullanılan 3×33 \times 33×3 komşuluk matrisi yapıları ayrı bir veri modeli ile temsil edildi. Bu kapsamda, filtre maskeleri KERNEL varlığında tanımlanmış, ilgili işlemler ile kernel yapıları arasındaki ilişki ise ISLEMKERNEL varlığı üzerinden kurulfu. Böylece mean, median ve kenar bulma gibi işlemlerde kullanılan maskelerin boyut ve tür bilgileri dinamik olarak yönetilebilir hale getirdik.

Görüntü işleme sürecinin izlenebilirliğini sağlamak amacıyla ISLEMUYGULAMA varlığı oluşturdukl. Bu yapı, bir görüntünün hangi işlemden geçerek yeni bir görüntüye dönüştüğünü giris_goruntu_id ve cikis_goruntu_id alanları üzerinden takip ediyor. Bu yaklaşım, sistemin kullanıcı kontrollü tek işlem prensibi ile uyumlu olup, her bir işlemin bağımsız olarak analiz edilmesini sağlıyor.

Sistemde kullanılan görüntülere ait meta veriler ise GORUNTU varlığı içerisinde tutuluyro. Bu varlık; görüntünün çözünürlüğü (M×NM \times NM×N), renk uzayı (RGB veya gri seviye) ve dosya yolu gibi teknik bilgileri içeriyor. Bu sayede görüntüler, NumPy matrisine dönüştürülmeden önce standart bir veri yapısında temsil edilr.

Ayrıca, proje kapsamında kullanıcı ve oturum takibi için KULLANICI, PROJEEKIPUYESI ve UYGULAMAOTURUMU varlıkları tanımladık. Bu yapılar, proje ekibinde yer alan kullanıcıların sistem üzerindeki etkileşimlerinin kronolojik olarak izlenmesini sağlıyor.

Sonuç olarak tasarladığımız ER diyagramı, görüntü verisi, uygulanan işlemler ve bu işlemlerin parametreleri arasındaki ilişkileri açık ve düzenli bir biçimde ortaya koyduğunu düşünüyrouz , sistemin hem veri bütünlüğünü korumasını hem de gerçekleştirilen işlemlerin geriye dönük analiz edilebilir olmasını sağlamaya çalıştık.



Uygulamanın Çalışma Prensibi

Uygulamamızda, kullanıcıdan alınan bir görüntünün piksel tabanlı olarak çalışmasını sağlamaya çalıştık . İlk aşamada görüntü, üç boyutlu bir matris yapısı olarak belleğe alınıp her bir piksele doğrudan erişim sağlayarakk işlemleri gerçekleştiriyor.

Görüntü üzerine uygulanacak işlemler, kullanıcı tarafından seçiliyor. Sistem yalnızca seçilen algoritmayı çalıştıracak şekilde tasarlandı bu yüzden . Gürültü ekleme aşamasında, rastgele seçilen pikseller siyah (0) ve beyaz (255) değerler ile değiştirerek Salt & Pepper gürültüsü oluşturup, ardından mean ve median filtreler kullanarak bu gürültüyü azalttık; median filtre bu tür gürültülerde daha etkili sonuçlar verdi. Kenar bulma işlemi, görüntüdeki yoğunluk değişimlerinin hesaplanması ile gerçekleştirilip Sobel operatörüne benzer gradyan maskeleri kullanılarak manuel konvolüsyon yöntemi ile uyguladık. Motion blur işlemini, belirli bir doğrultuda oluşturulan kernel matrisi yardımıyla görüntü üzerinde kaydırma etkisi oluşturacak şekilde gerçekleştirdik, morfolojik işlemler kapsamında genişleme (dilation) ve aşınma (erosion) işlemleri komşu piksellerin maksimum ve minimum değerlerinin seçilmesine dayandı bu sayede ; açma ve kapama işlemleri ise bu temel işlemlerin birleşimi ile elde edildi.

Tüm bu işlemlerde, herhangi bir hazır görüntü işleme fonksiyonu kullanmadık, doğrudan piksel değerleri üzerinde gerçekleştirilen matematiksel hesaplamalar ile kendimiz oluşturduk.

Kaynaklar

1)Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing* (4th ed.). Pearson Education.
→ Zaten mevcut, temel görüntü işleme ve matematiksel modeller için.

2)Szeliski, R. (2022). *Computer Vision: Algorithms and Applications* (2nd ed.). Springer.
→ Geometrik dönüşümler, konvolüsyon ve kenar bulma mantığı için referans.

3)Pratt, W. K. (2007). *Digital Image Processing: PIKS Scientific Inside* (4th ed.). Wiley- Interscience.
→ Filtreleme ve morfolojik işlemler hakkında detaylı açıklamalar.

4)Jain, A. K. (1989). *Fundamentals of Digital Image Processing*. Prentice Hall.
→ Histogram, eşikleme ve piksel tabanlı işlemler için temel literatür.

5)Gonzalez, R. C., & Woods, R. E. (2018). Digital Image Processing, Chapter 3: Intensity Transformations and Spatial Filtering. Pearson Education.
→ Kontrast germe, yoğunluk dönüşümleri ve filtreler için spesifik bölüm.

6)Castleman, K. R. (1996). Digital Image Processing. Prentice Hall.
→ Gradyan ve kenar bulma algoritmaları hakkında klasik referans.

7)Burger, W., & Burge, M. J. (2016). Digital Image Processing: An Algorithmic Introduction Using Java. Springer.
→ Piksel bazlı algoritmaların mantığını anlatan uygulamalı kitap.

Online Kaynaklar

8) SciPy Documentation. (2025). https://docs.scipy.org/
→ NumPy ve sinyal/konvolüsyon referansı için

9) Stack Overflow & GitHub örnekleri (2025).
→ Manuel filtreleme ve komşuluk hesaplama örnekleri

10) OpenCV Documentation. (2025). Retrieved from https://docs.opencv.org/

11) NumPy Documentation. (2025). Retrieved from https://numpy.org/doc/

12) Matplotlib Documentation. (2025). Retrieved from https://matplotlib.org/stable/index.html

13) Pillow (PIL) Documentation. (2025). Retrieved from https://pillow.readthedocs.io/

14) Python Software Foundation. (2025). *Tkinter Documentation*. Retrieved from https://docs.python.org/3/library/tkinter.html

NumPy kısmı için: [8, 11]
OpenCV I/O süreci için: [10]
Arayüz (Tkinter) kısmı için: [14]

Ek Not

Bu projede kullanılan tüm algoritmalar (gri dönüşüm, eşikleme, konvolüsyon, kenar bulma vb.), literatürde yer alan matematiksel modellere dayanarak kendi emeğimizle , herhangi bir hazır görüntü işleme fonksiyonu kullanılmadan gerçekleştirdik.